Cloud Computing Course (Beginner to Advanced)

Course Structure

Phase 1: Cloud Fundamentals (Beginner)

1. What is Cloud Computing?
2. Why do we need Cloud Computing?
3. Problems with Traditional Computing
4. Types of Cloud Services (IaaS, PaaS, SaaS)
5. Cloud Deployment Models
6. Virtualization
7. Virtual Machines
8. Hypervisors

Phase 2: Core Cloud Concepts

9. Data Centers
10. Networking Basics
11. Storage
12. Compute
13. Load Balancer
14. Auto Scaling

Phase 3: AWS Fundamentals

15. AWS Introduction
16. IAM
17. EC2
18. EBS
19. S3
20. VPC
21. RDS
22. Route 53

Phase 4: Advanced Cloud

23. Docker
24. Kubernetes
25. Serverless (Lambda)
26. Monitoring
27. Security
28. CI/CD
29. Terraform
30. Real-world Projects

Lesson 1: What is Cloud Computing?

Before learning cloud computing, let's first understand **computing**.

What is Computing?

Computing means using a computer to perform tasks like:

- Storing data
- Processing data
- Running applications
- Accessing the internet
- Solving problems

For example, when you:

- Watch YouTube
- Use WhatsApp
- Shop on Amazon
- Send an email

A computer somewhere is processing your request.

What is Cloud?

The word **Cloud** doesn't mean the clouds in the sky.

In computer science, **Cloud** refers to **servers and services that are available over the Internet**.

Think of it like this:

Your Laptop



| Storage |

| Database |

| Networking |

| Applications |

+----------------------+

Instead of storing everything on your own computer, you use powerful computers owned by cloud providers.

What is Cloud Computing?

Definition:

Cloud Computing is the delivery of computing resources (servers, storage, databases, networking, software, etc.) over the Internet, where users pay only for the resources they use.

In simple words:

Instead of buying your own servers, you rent them from a cloud provider.

Real-Life Example: Electricity

Imagine you want electricity in your house.

You have two options:

Option 1: Build Your Own Power Plant

You would need:

- Land
- Machines
- Engineers
- Fuel
- Maintenance

This is expensive and difficult.

Option 2: Use the Electricity Board

You simply:

- Connect your house.
- Use electricity.
- Pay the monthly bill.

Cloud computing works the same way.

Instead of building your own data center, you rent computing resources and pay only for what you use.

Another Example: Water Supply

Would you build your own water treatment plant?

No.

You simply use water from the municipal supply.

Cloud computing follows the same idea:

- Someone else manages the infrastructure.
- You use the service.

Traditional Computing vs Cloud Computing

Imagine you want to launch an e-commerce website.

Traditional Computing

You need to buy:

Server

CPU

RAM

Storage

Network Devices

UPS

Cooling System

Backup System

Security Devices

If your website becomes popular, you may need to buy even more hardware.

Cloud Computing

You create an account with a cloud provider.

Within minutes, you can rent:

- A virtual server
- Storage
- A database
- Networking
- Security services

If your traffic increases, you can increase your resources with just a few clicks.

Real-World Scenario

Imagine your college is organizing online admissions.

On normal days:

100 students visit the website.

On the last day:

50,000 students visit the website.

Traditional Setup

The server may become overloaded, causing the website to slow down or crash.

Cloud Setup

The cloud can automatically provide more computing resources to handle the extra traffic, then reduce them when demand drops.

This ability to increase or decrease resources is called **scalability**, and we'll study it in detail later.

Everyday Cloud Services

You already use cloud computing every day.

Service	Cloud Resource Used
Gmail	Email servers and storage
Google Drive	Cloud storage
OneDrive	Cloud storage
Netflix	Cloud servers and streaming
Spotify	Cloud streaming
Amazon	Cloud servers and databases
Instagram	Cloud storage and databases
WhatsApp	Cloud messaging infrastructure

Even though you don't see the servers, they are running in cloud data centers.

Why is Cloud Computing Popular?

Companies choose cloud computing because it offers:

- Lower upfront costs
- Fast deployment
- Easy scaling
- High availability
- Global access
- Built-in backup options
- Security features
- Managed services

What Can You Rent from the Cloud?

Cloud providers let you rent many types of resources:

- Virtual servers (compute)
- Storage
- Databases
- Networking
- Artificial Intelligence services
- Machine Learning platforms
- Analytics tools
- Security services
- Monitoring tools

Key Terms Introduced Today

Term	Meaning
Computing	Using computers to process and store information
Cloud	Computing resources available over the Internet
Cloud Computing	Renting computing resources over the Internet
Server	A computer that provides services to other computers
Internet	The network that connects users to cloud services

Summary

Today you learned:

- What computing means
- What "cloud" means in technology
- What cloud computing is
- Why cloud computing is useful

- How it differs from traditional computing
- Real-life examples of cloud computing
- Services you already use that rely on the cloud

Practice Questions

Try answering these without looking back:

1. What is computing?
2. What is a cloud in cloud computing?
3. Define cloud computing in your own words.
4. How is cloud computing different from traditional computing?
5. Give three examples of cloud services you use every day.
6. Why do companies prefer cloud computing?

Homework

Think about this question:

When you upload a photo to Google Drive from your phone, where is the photo actually stored?

Don't just answer "in the cloud." Try to explain **what "the cloud" really is** based on today's lesson.

Lesson 2: Why Do We Need Cloud Computing?

Imagine This Situation

Suppose you are starting an online shopping website.

At first, you have only **100 customers per day**.

So you buy one server.

Customers

```
    |
    ▼
+----------------+
| Physical Server|
+----------------+
```

Everything works well.

After One Year

Your business grows.

Now you have:

- 10,000 customers per day
- Thousands of products
- Many online payments

Your single server becomes overloaded.

Customers

```
↓↓↓↓↓↓↓↓↓↓↓

+----------------+
| Physical Server|
| 100% CPU       |
| 100% RAM       |
```

+----------------+

The website becomes slow.

Some users cannot even open it.

What Can You Do?

You have two choices.

Option 1: Buy a Bigger Server

You purchase:

- More CPU
- More RAM
- More Storage

Problems:

- Expensive
- Takes days or weeks to install
- If traffic increases again, you'll have to upgrade again

Option 2: Buy Multiple Servers

Now you have:

Server 1

Server 2

Server 3

Server 4

But now you need:

- Network engineers
- Hardware engineers
- Cooling systems
- Electricity

- Security
- Backup systems

Your costs increase significantly.

The Biggest Problem

Imagine it's the end of the festival sale.

Traffic drops.

Instead of 10,000 users,

you now have only **500 users**.

But your servers are still running.

4 Powerful Servers

Only 5% Used

You're paying for servers that are mostly idle.

This is a waste of money.

Another Real-Life Example

Imagine you own a bus.

The bus has **50 seats**.

Morning

50 passengers

Perfect.

Afternoon

Only 2 passengers

The bus still consumes:

- Fuel
- Driver salary

- Maintenance

You're paying for unused capacity.

Servers work the same way.

Cloud Computing Solves This Problem

Instead of buying servers,

you rent them only when you need them.

Normal Day

1 Virtual Server

Festival Sale

10 Virtual Servers

After Sale

Back to 1 Virtual Server

You only pay for the servers while you use them.

Main Problems Before Cloud Computing

Problem 1: High Cost

Buying hardware is expensive.

You need:

- Server
- CPU
- RAM
- Storage
- UPS
- Cooling
- Networking
- Security

Cloud removes the large upfront investment.

Problem 2: Scalability

Imagine your website suddenly becomes popular.

Traditional setup:

Need Bigger Server

↓

Order Hardware

↓

Wait 2 Weeks

Cloud:

Click

↓

New Server Ready in Minutes

Problem 3: Maintenance

Traditional setup requires:

- Hardware repairs
- Disk replacements
- Power monitoring
- Cooling systems

- Operating system updates

Cloud providers handle much of the underlying infrastructure.

Problem 4: Backup

What if your server catches fire?

Server Destroyed

↓

Data Lost

With cloud services, you can configure backups and even replicate data across multiple locations.

Problem 5: Availability

Suppose your server is located only in Chennai.

A user in the USA accesses your website.

USA

↓

Internet

↓

India

↓

Server

The greater the distance, the higher the latency.

Cloud providers have data centers around the world, allowing applications to serve users from locations closer to them.

Problem 6: Disaster Recovery

Imagine an earthquake damages your data center.

Traditional setup:

Website goes offline until systems are restored.

Cloud setup:

Applications can often fail over to another region if designed for it.

Why Companies Love the Cloud

Cloud computing offers:

- Lower initial costs
- Easy scalability
- Faster deployment
- High availability
- Global reach
- Flexible resource usage
- Managed infrastructure
- Backup and disaster recovery options

Real Company Example: Netflix

Imagine millions of people start watching a new movie on Friday night.

Traffic increases dramatically.

Netflix uses cloud infrastructure to scale resources so users can continue streaming smoothly.

Important Cloud Concepts Introduced Today

1. Scalability

The ability to increase or decrease computing resources as demand changes.

Example:

100 Users

↓

1 Server

10,000 Users

↓

10 Servers

---

2. Pay-as-You-Go

You pay only for the resources you use.

Like paying an electricity bill based on consumption.

---

3. On-Demand Resources

Need a server?

Create one in minutes.

Need more storage?

Add it immediately.

No need to purchase new hardware.

Summary

Cloud computing solves many limitations of traditional infrastructure:

- High hardware costs
- Slow scaling
- Complex maintenance
- Limited backup options
- Disaster recovery challenges
- Global accessibility

Interview Questions

1. Why was cloud computing introduced?
2. What problems does cloud computing solve?
3. What is scalability?
4. What is the pay-as-you-go model?
5. Why is cloud computing cheaper for many businesses?
6. What is disaster recovery?
7. Why is high availability important?

Quick Check

Imagine you're building a website.

On normal days:

- 500 users

During a sale:

- 50,000 users

Question: Would you rather:

- Buy 20 physical servers that sit mostly idle after the sale, or
- Rent extra cloud servers only during the sale and remove them afterward?

Lesson 3: The 5 Essential Characteristics of Cloud Computing

Question: How do we know if a service is truly a cloud service?

The answer comes from **NIST (National Institute of Standards and Technology)**, which defines **5 essential characteristics** of cloud computing.

Think of these as the **five rules** that make cloud computing "cloud."

The 5 Characteristics

Cloud Computing

1. On-Demand Self-Service
2. Broad Network Access
3. Resource Pooling
4. Rapid Elasticity
5. Measured Service

We'll understand each one with a real-life example.

---

1. On-Demand Self-Service

What does it mean?

You can create, modify, or delete cloud resources **whenever you want**, without calling or emailing the cloud provider.

You are in control.

Real-Life Example

Imagine you're hungry.

Option 1:
You call a restaurant.

They say:

"We'll send someone tomorrow."

Not convenient.

Option 2:
You open a food delivery app.

Choose food.

Pay.

Food arrives.

You didn't need permission from anyone.

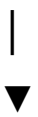
Cloud computing works in a similar way.

Cloud Example

Suppose you need a server.

Traditional way:

Employee



IT Department



Manager Approval



Server after 2 weeks

Cloud way:

Login to AWS

↓

Click "Launch Instance"

↓

Server Ready in 2 Minutes

You don't need to wait for someone to install hardware.

Example

Need:

- 1 Server
- 20 GB Storage
- Database

You simply create them from the cloud portal.

That's **On-Demand Self-Service**.

---

2. Broad Network Access

What does it mean?

Cloud services are accessible **over the Internet** from many types of devices.

For example:

- Laptop
- Desktop
- Mobile phone
- Tablet

As long as you have internet access and permission, you can use the service.

Example

You upload a photo to Google Drive using your phone.

Later, you open your laptop.

You log in.

The same photo appears.

Phone

Internet

Cloud Storage

Laptop

The cloud isn't tied to one device.

---

3. Resource Pooling

This is one of the most important concepts.

What does it mean?

A cloud provider owns thousands of physical servers.

These resources are **shared securely** among many customers.

Example

Imagine a hotel.

Hotel

Room 101

Room 102

Room 103

Room 104

Different guests stay in different rooms.

They all use the same building.

The building is shared, but each guest has their own private room.

Cloud Example

AWS owns one physical server.

Physical Server

CPU

RAM

Storage

It creates virtual machines for different customers.

Customer A

Ubuntu VM

Customer B

Windows VM

Customer C

Linux VM

Everyone shares the same physical hardware.

But nobody can see another customer's data.

This sharing is possible because of **virtualization**, which we learned earlier.

Why Resource Pooling?

It helps providers:

- Save money
- Use hardware efficiently
- Serve millions of customers

---

4. Rapid Elasticity

This is another very important concept.

What does it mean?

Cloud resources can be increased or decreased **quickly** as demand changes.

Example

Imagine a shopping mall.

Monday:

100 Customers

↓

1 Cash Counter

Festival Day:

20,000 Customers

↓

20 Cash Counters

After the festival:

100 Customers

↓

1 Cash Counter

The number of counters changes based on demand.

Cloud servers work the same way.

Website Example

Normal day:

100 Users

↓

1 Server

During a sale:

100,000 Users

↓

20 Servers

After the sale:

100 Users

↓

1 Server

Resources grow and shrink automatically.

This ability is called **Rapid Elasticity**.

---

5. Measured Service

What does it mean?

The cloud measures how much you use.

You pay only for what you consume.

Example

Electricity Bill

You don't pay a fixed amount every month.

You pay according to:

- Units consumed

Similarly, cloud providers measure things like:

- CPU usage
- Storage used
- Data transferred
- Running time of virtual machines

Example

Suppose you use:

- One server for 10 hours
- 50 GB storage

You pay only for those resources.

If you stop the server, charges for compute usually stop as well (though storage may still incur charges depending on the service).

This is known as the **pay-as-you-go** model.

Summary Table

Characteristic	Simple Meaning	Real-Life Example
On-Demand Self-Service	Create resources yourself	Ordering food with an app
Broad Network Access	Access from anywhere with internet	Google Drive on phone and laptop
Resource Pooling	Shared physical infrastructure	Hotel with many rooms
Rapid Elasticity	Scale resources up or down quickly	Opening more billing counters during a sale
Measured Service	Pay only for what you use	Electricity bill

Memory Trick

Remember this sequence:

S A P E M

- **S** → Self-Service
- **A** → Access
- **P** → Pooling
- **E** → Elasticity
- **M** → Measured Service

Real Example: Google Drive

When you upload a file:

✅ You upload it yourself → On-Demand Self-Service

✅ You can access it from your phone or laptop → Broad Network Access

✅ Google stores your file on shared infrastructure → Resource Pooling

✅ Google can add more storage systems as demand grows → Rapid Elasticity

✅ Your storage usage is tracked, and paid plans are billed based on usage → Measured Service

One service demonstrates all five characteristics.

Interview Questions

1. What are the five essential characteristics of cloud computing?
2. What is Resource Pooling?
3. Explain Rapid Elasticity with an example.
4. What is Measured Service?
5. What is Broad Network Access?
6. Why is On-Demand Self-Service important?
7. Which characteristic enables the pay-as-you-go model?

Practice Exercise

Suppose you launch a video streaming website.

One day:

- 500 users

During a cricket final:

- 2,000,000 users

Try to answer:

1. Which cloud characteristic allows your servers to increase automatically?
2. Which characteristic allows users to watch from phones, TVs, and laptops?
3. Which characteristic lets you create new servers without contacting the provider?
4. Which characteristic means you pay only for the resources consumed?
5. Which characteristic allows cloud providers to share physical servers among many customers while keeping each customer's environment isolated?

Lesson 4: Cloud Service Models (IaaS, PaaS, SaaS)

First, Understand the Big Picture

Imagine you want to **open a restaurant**.

There are three ways to do it.

Option 1: Build Everything Yourself

You:

- Buy land
- Construct the building
- Buy kitchen equipment
- Hire staff
- Manage electricity
- Cook the food

You control everything.

Option 2: Rent a Ready Kitchen

Someone else provides:

- Building
- Kitchen
- Electricity
- Water

You only cook and sell food.

Less work.

Option 3: Order Food Online

You simply eat.

Someone else does everything.

Almost no management.

Cloud computing works exactly like this.

The three cloud service models differ in **who manages what**.

The Three Service Models

Cloud Computing

SaaS

▲

PaaS

▲

IaaS

As you move upward:

- You manage **less**
- The cloud provider manages **more**

Before Understanding the Models

Let's see what an application actually consists of.

Suppose you create a shopping website.

It needs:

Shopping Website

Application Code

↓

Runtime (Java, Python, Node.js)

↓

Operating System

↓

Virtual Machine

↓

Physical Server

↓

Storage

↓

Networking

Someone has to manage every layer.

The question is:

Who manages which layer?

---

1. IaaS (Infrastructure as a Service)

Definition

The cloud provider gives you:

- Virtual Server

- Storage
- Networking

You manage:

- Operating System
- Runtime
- Applications
- Data

Diagram

You Manage

Application

Runtime

Operating System

Cloud Provider Manages

Virtual Machine

Storage

Networking

Physical Server

Example

Suppose AWS gives you an EC2 instance.

Inside it you install:

- Ubuntu
- Java
- Python
- MySQL
- Your website

AWS only provides the infrastructure.

Everything inside the server is your responsibility.

Real-Life Example

Imagine renting an empty apartment.

The owner provides:

- Building
- Electricity
- Water

You bring:

- Furniture
- TV
- Bed
- Refrigerator

Exactly like IaaS.

Popular IaaS Services

- AWS EC2
- Azure Virtual Machines
- Google Compute Engine

---

2. PaaS (Platform as a Service)

Definition

The provider manages:

- Infrastructure
- Operating System
- Runtime

You only upload your application.

Diagram

You Manage

Application

Data

Cloud Provider Manages

Runtime

Operating System

Virtual Machine

Storage

Networking

Physical Server

Example

Suppose you write a Node.js application.

Instead of installing:

- Ubuntu
- Node.js
- Nginx

You simply upload your code.

The cloud platform runs it.

Real-Life Example

Imagine renting a fully equipped kitchen.

Everything is already there.

You only cook.

Popular PaaS Services

- Heroku
- Google App Engine
- Azure App Service
- AWS Elastic Beanstalk

---

3. SaaS (Software as a Service)

Definition

The provider manages everything.

You simply use the software.

Example

When you use Gmail,

Do you install:

- Servers?
- Operating systems?

- Databases?

No.

You simply log in and use it.

Diagram

Cloud Provider Manages

Application

Runtime

Operating System

Virtual Machine

Storage

Networking

Physical Server

You

Use the Software

Real-Life Example

Ordering food from a restaurant.

You don't cook.

You simply eat.

Popular SaaS Services

- Gmail
- Google Drive

- Microsoft 365
- Dropbox
- Zoom
- Slack
- Canva

Comparison Table

Layer	IaaS	PaaS	SaaS
Application	You	You	Provider
Data	You	You	Provider (with user responsibility for content)
Runtime	You	Provider	Provider
Operating System	You	Provider	Provider
Virtual Machine	Provider	Provider	Provider
Storage	Provider	Provider	Provider
Networking	Provider	Provider	Provider
Physical Server	Provider	Provider	Provider

Easy Memory Trick

Imagine you need transportation.

IaaS → Rent a Car 🚙

The company gives you the car.

You:

- Drive it
- Fill fuel
- Decide the route

You have maximum control.

PaaS → Taxi 🚕

Driver is included.

You simply tell the destination.

Less work.

SaaS → Bus 🚌

Everything is managed.

You simply sit and travel.

Very little control.

Another Real Example

Suppose you are developing a website.

IaaS

AWS says:

"Here is your virtual machine."

You install:

- Ubuntu
- Node.js
- MySQL
- Nginx
- Your application

PaaS

Heroku says:

"Upload your code."

That's all.

SaaS

Google says:

"Use Gmail."

No installation required.

Which One Gives More Control?

More Control

IaaS

↓

PaaS

↓

SaaS

Less Control

Which One Requires More Work?

More Work

IaaS

↓

PaaS

↓

SaaS

Less Work

Which One Requires More Technical Knowledge?

Highest

IaaS

↓

PaaS

↓

SaaS

Lowest

Interview Questions

1. What is IaaS?

Infrastructure as a Service provides virtualized computing resources like servers, storage, and networking. You manage the operating system, runtime, and applications.

---

2. What is PaaS?

Platform as a Service provides a managed platform for developing and deploying applications. You focus on your code while the provider manages the infrastructure and runtime.

---

3. What is SaaS?

Software as a Service delivers complete software over the internet. Users simply access and use the application.

---

4. Give examples.

- IaaS → AWS EC2
- PaaS → Heroku
- SaaS → Gmail

---

5. Which service model gives maximum control?

Answer: IaaS

---

6. Which requires the least management?

Answer: SaaS

Quick Quiz

Imagine you're creating a Python web application.

Scenario 1

AWS gives you an empty virtual machine.

You install Python, Flask, and MySQL yourself.

👉 Which model is this?

Answer: IaaS

Scenario 2

You upload your Flask code to a platform, and it automatically runs.

👉 Which model?

Answer: PaaS

Scenario 3

You simply log into Gmail to send emails.

👉 Which model?

Answer: SaaS

Summary

Service Model	You Manage	Provider Manages	Example
IaaS	OS, runtime, apps, data	Infrastructure	AWS EC2
PaaS	Apps, data	Platform + infrastructure	Heroku
SaaS	Use the software	Everything	Gmail

Lesson 5: Data Centers, Regions, and Availability Zones (AZs)

This lesson explains **where the cloud actually exists**. Many beginners think the cloud is something abstract. In reality, cloud services run on **real computers inside real buildings**.

---

1. What is a Data Center?

A **Data Center** is a large building that contains thousands of physical servers along with networking equipment, storage systems, power supplies, and cooling systems.

These servers provide cloud services to customers.

Think of it like this

Imagine a huge library.

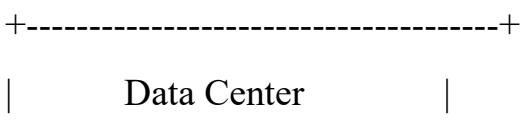
- The building = Data Center
- Bookshelves = Server racks
- Books = Data stored on servers

Just as a library stores books, a data center stores and processes digital information.

Inside a Data Center

A typical data center contains:

- Thousands of servers
- Storage devices
- Network switches and routers
- Backup power (UPS and generators)
- Cooling systems
- Fire protection
- Security systems



```
|                                    |
| Server Rack   Server Rack          |
| Server Rack   Server Rack          |
| Storage       Networking           |
| Cooling       Power Backup         |
+------------------------------------+
```

---

2. Why Do Cloud Providers Need Data Centers?

Suppose millions of people are using:

- Gmail
- YouTube
- Netflix
- Amazon
- WhatsApp

All of these applications require powerful computers running 24/7.

Your laptop cannot serve millions of users.

Cloud providers build massive data centers to handle this demand.

---

3. What is a Region?

A **Region** is a **geographical area** where a cloud provider has one or more data centers.

For example:

- Mumbai
- Singapore
- London
- Tokyo
- Frankfurt
- Virginia

Each region is physically located in a different part of the world.

World

├── India (Mumbai Region)
├── Singapore Region
├── London Region
├── Tokyo Region
└── Virginia Region

Why Multiple Regions?

Imagine your website's users are in India.

If your server is in the United States, every request has to travel a long distance, increasing response time (latency).

User (India)

    |

Internet

    |

USA Server

If the server is in Mumbai:

User (India)

    |

Internet

    |

Mumbai Server

The response is much faster.

---

4. What is an Availability Zone (AZ)?

An **Availability Zone (AZ)** is one or more physically separate data centers within the same region.

For example:

Mumbai Region

- AZ-1
- AZ-2
- AZ-3

Mumbai Region

├── AZ-1
├── AZ-2
└── AZ-3

Each Availability Zone has:

- Independent power
- Independent cooling
- Independent networking

This helps improve reliability.

Why Multiple Availability Zones?

Imagine all your servers are in one building.

If that building loses power, your application goes offline.

Now imagine your application runs in two different Availability Zones.

Mumbai Region

AZ-1 ---> Application

AZ-2 ---> Backup Application

If AZ-1 has a problem, traffic can be redirected to AZ-2.

This increases **High Availability**.

---

5. Region vs Availability Zone

Region	Availability Zone
Large geographical location	Separate data center(s) inside a region
Example: Mumbai	Example: Mumbai AZ-1
Contains multiple AZs	Contains servers and infrastructure

---

6. Real AWS Example

Suppose AWS has:

Asia Pacific (Mumbai)

├── Availability Zone A
├── Availability Zone B
└── Availability Zone C

If you launch an EC2 instance, you choose:

- Region: Mumbai
- Availability Zone: A, B, or C

---

7. Why Do Companies Use Multiple AZs?

Imagine you run an online banking application.

If one Availability Zone fails:

- Customers should still be able to access their accounts.
- Transactions should continue.

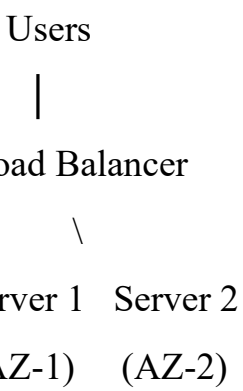
To achieve this, companies deploy applications across multiple AZs.

---

8. High Availability

High Availability (HA) means keeping an application running even if part of the infrastructure fails.

Example:



If Server 1 fails, the Load Balancer sends traffic to Server 2.

---

9. Disaster Recovery

A disaster could affect an entire region.

For example:

- Flood
- Earthquake
- Major power outage

To prepare for this, companies often replicate data and applications to another region.



If Mumbai becomes unavailable, services can continue from Singapore.

---

10. Key Terms

Term	Meaning
Data Center	Building containing servers and networking equipment
Region	Geographic area containing multiple Availability Zones
Availability Zone	One or more isolated data centers within a region
High Availability	Keeping services running despite failures
Disaster Recovery	Restoring services after a major outage

Memory Trick

Think of it like this:

- **Country** → Region
- **City** → Availability Zone
- **House** → Data Center

So:

Region

↓

Availability Zone

↓

Data Center

↓

Servers

Interview Questions

1. What is a data center?
2. What is a cloud region?
3. What is an Availability Zone?
4. Why does AWS have multiple Availability Zones in one region?
5. What is the difference between a Region and an Availability Zone?

6. What is High Availability?
7. Why do companies use multiple regions?

Practice Exercise

Imagine you are building an online shopping website for users across India.

Answer these questions:

1. Would you choose a region in Mumbai or London? Why?
2. Would you deploy your application in one Availability Zone or multiple Availability Zones?
3. What happens if one Availability Zone fails?
4. Why is disaster recovery important for an e-commerce website?

Next Lesson: Virtualization and Hypervisors (Deep Dive)

We'll explore:

- How one physical server runs many Virtual Machines
- Type 1 vs Type 2 Hypervisors
- Virtualization vs Containers
- How AWS EC2 uses virtualization internally
- Resource allocation (CPU, RAM, Storage)
- Practical VM creation examples and interview questions

next

Excellent! Before we learn **AWS EC2**, you must understand **Virtualization** deeply because **EC2 is built on virtualization**.

This lesson will explain **exactly what happens inside a physical server** when you create a Virtual Machine.

Lesson 6: Virtualization and Hypervisors (Deep Dive)

Learning Objectives

By the end of this lesson, you'll understand:

- What virtualization really is
- Why virtualization is needed
- How one physical server runs multiple VMs
- What a hypervisor does
- Type 1 vs Type 2 hypervisors
- How AWS uses virtualization

Step 1: The Problem Before Virtualization

Imagine you buy a powerful server.

Specifications:

- CPU: 32 Cores
- RAM: 128 GB
- Storage: 2 TB

Now you install only one application.

```
+-------------------------------+
| Physical Server               |
| CPU: 32 Cores                 |
| RAM: 128 GB                   |
| Storage: 2 TB                 |
|                               |
| One Application Running       |
+-------------------------------+
```

The application uses only:

- 2 CPU cores
- 8 GB RAM
- 100 GB storage

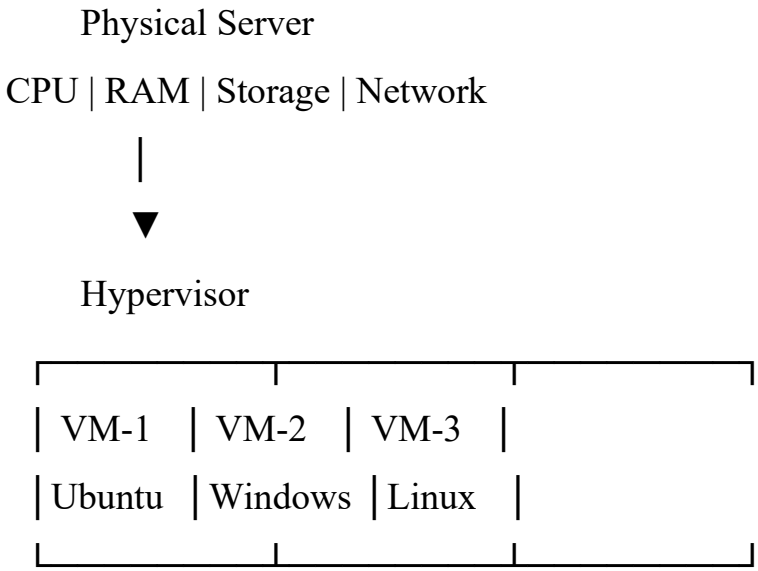
The remaining hardware sits idle.

Resource	Total	Used
CPU	32 Cores	2 Cores
RAM	128 GB	8 GB
Storage	2 TB	100 GB

This is **poor resource utilization**.

Step 2: The Solution — Virtualization

Instead of one operating system, we create multiple Virtual Machines (VMs).



Now the same physical server can run multiple operating systems simultaneously.

Step 3: What is a Virtual Machine?

A **Virtual Machine (VM)** is a software-based computer.

It behaves exactly like a physical computer.

A VM has:

- Virtual CPU (vCPU)
- Virtual RAM

- Virtual Disk
- Virtual Network Card
- Operating System

Example:

VM-1

Operating System : Ubuntu

CPU : 4 vCPUs

RAM : 8 GB

Disk : 100 GB

You can install software, browse the internet, create files, and run applications just as you would on a physical machine.

Step 4: What is a Hypervisor?

A **Hypervisor** is software that creates, manages, and isolates Virtual Machines.

Think of it as the **manager** of the physical server.

It decides:

- How much CPU each VM gets
- How much RAM each VM gets
- How much storage each VM gets
- Which VM can access which resources

Without a hypervisor, virtualization would not be possible.

Real-Life Analogy

Imagine a hotel.

- Hotel building = Physical Server
- Rooms = Virtual Machines
- Guests = Applications

- Hotel manager = Hypervisor

The hotel manager ensures:

- Every guest gets the correct room.
- Guests cannot enter each other's rooms.
- Resources are allocated fairly.

Similarly, the hypervisor keeps each VM isolated and manages resource allocation.

Step 5: Resource Allocation

Suppose a physical server has:

- 16 CPU cores
- 64 GB RAM

The hypervisor might allocate resources like this:

VM	CPU	RAM
VM-1	4 Cores	16 GB
VM-2	4 Cores	16 GB
VM-3	4 Cores	16 GB
VM-4	4 Cores	16 GB

Each VM believes it owns those resources, even though they're all sharing the same physical server.

Step 6: Isolation Between VMs

Suppose:

- VM-1 is running a web server.
- VM-2 is running a database.
- VM-3 is being used for testing.

If VM-3 crashes due to a software bug:

VM-1 : Running ✅

VM-2 : Running ✅

VM-3 : Crashed ❌

The other VMs continue working because each VM is isolated from the others.

This isolation improves reliability and security.

Step 7: Types of Hypervisors

Type 1 Hypervisor (Bare Metal)

Runs directly on the physical hardware.

Applications

|

Operating Systems

|

Hypervisor

|

Physical Hardware

Examples:

- VMware ESXi
- Microsoft Hyper-V
- KVM
- Xen

Advantages

- High performance
- Better security
- Used in cloud data centers

Type 2 Hypervisor (Hosted)

Runs on top of an existing operating system.

Applications

Virtual Machines

Hypervisor

Host Operating System

Physical Hardware

Examples:

- Oracle VirtualBox
- VMware Workstation

Advantages

- Easy to install
- Great for learning and development

Type 1 vs Type 2

Feature	Type 1	Type 2
Runs on	Physical hardware	Host operating system
Performance	Higher	Lower
Primary use	Cloud providers and data centers	Personal computers and testing

---

Step 8: How AWS Uses Virtualization

Suppose you launch an EC2 instance.

Here's what happens:

1. You choose an instance type (for example, 2 vCPUs and 4 GB RAM).

2. AWS selects a physical server with available resources.
3. The hypervisor creates a new Virtual Machine.
4. CPU, RAM, storage, and networking are allocated.
5. The selected operating system (Ubuntu, Windows, etc.) starts.
6. You connect to the VM using SSH (Linux) or RDP (Windows).

From your perspective, it feels like your own dedicated server.

Step 9: Virtual Machines vs Physical Servers

Physical Server	Virtual Machine
Real hardware	Software-based computer
One operating system (typically)	Multiple VMs can share one server
Expensive to scale	Easy to create and remove
Slower provisioning	Fast provisioning

Step 10: Virtual Machines vs Containers

Virtual Machine	Container
Includes a full operating system	Shares the host OS kernel
Larger	Lightweight
Slower startup	Starts quickly
Uses more resources	Uses fewer resources

We'll study Docker and Kubernetes later in the course.

Real-World Example

Imagine you're developing an application that must work on:

- Ubuntu
- Windows

- CentOS

Without virtualization, you'd need three physical computers.

With virtualization:

Laptop

Hypervisor

Ubuntu VM

Windows VM

CentOS VM

One laptop can run all three operating systems.

Summary

- **Virtualization** allows one physical server to run multiple Virtual Machines.
- A **Virtual Machine (VM)** behaves like a real computer.
- A **Hypervisor** creates and manages VMs.
- **Type 1 hypervisors** run directly on hardware and are commonly used by cloud providers.
- **Type 2 hypervisors** run on top of a host operating system and are commonly used for development and testing.
- AWS EC2 uses virtualization to provide virtual servers to customers.

Interview Questions

1. What is virtualization?
2. Why is virtualization important in cloud computing?
3. What is a Virtual Machine?
4. What is a hypervisor?
5. What is the difference between Type 1 and Type 2 hypervisors?

6. Why can multiple VMs run on one physical server?
7. How does AWS EC2 use virtualization?

Practice Exercise

Suppose a physical server has:

- 32 CPU cores
- 128 GB RAM

Design a virtualization plan for **4 Virtual Machines** by deciding how many CPU cores and how much RAM each VM should receive. Then explain why your allocation makes sense.

Lesson 7: Networking Fundamentals for Cloud Computing

What is a Network?

Definition

A **network** is a group of devices connected together so they can communicate and share data.

Devices can include:

- Computers
- Mobile phones
- Servers
- Printers
- Smart TVs

Real-Life Example

Imagine your college classroom.

There are 30 students.

If none of the students can talk to each other, they are isolated.

Now imagine everyone can communicate.

That is similar to a **network**.

Student A ⟷ Student B

↑ ↓

Student C ⟷ Student D

Similarly, computers communicate over a network.

Types of Networks

1. LAN (Local Area Network)

A LAN connects devices in a small area.

Examples:

- Home Wi-Fi
- School computer lab
- Office network

Home

Laptop
  |
Phone ── Wi-Fi Router ── Smart TV
  |
Desktop

All these devices are on the same local network.

---

2. MAN (Metropolitan Area Network)

A MAN connects multiple LANs within a city.

Example:

- A university with several campuses in one city.

---

3. WAN (Wide Area Network)

A WAN connects networks over large distances.

The biggest WAN is the **Internet**.

India ←———————— Internet ————————→ USA

Cloud providers operate over WANs because users connect from all over the world.

What is the Internet?

The **Internet** is a global network that connects millions of smaller networks.

Think of it as a worldwide highway.

Your Laptop



Internet



AWS Server

When you visit a website, your request travels across the Internet to a server.

What is an IP Address?

Every device connected to a network needs a unique identifier.

This identifier is called an **IP Address (Internet Protocol Address)**.

Think of an IP address like a **house address**.

Without an address, the post office wouldn't know where to deliver a letter.

Similarly, without an IP address, data wouldn't know where to go.

Example

Laptop

IP Address

192.168.1.10

Another computer:

Desktop

IP Address

192.168.1.20

Each device has its own address.

Why Do We Need IP Addresses?

Suppose your laptop wants to send data to another computer.

It needs to know **where to send it**.

Example:

Laptop

192.168.1.10

↓

Message

↓

Desktop

192.168.1.20

The IP address tells the network the destination.

Public IP vs Private IP

This is a very important interview topic.

Private IP

A **Private IP** is used **inside a local network**.

Example:

Home Wi-Fi

Laptop

192.168.1.10

Phone

192.168.1.20

These addresses are **not directly accessible from the Internet**.

Public IP

A **Public IP** is visible on the Internet.

For example:

Your Home Router

Public IP

49.xxx.xxx.xxx

When someone on the Internet connects to your home network, they reach your router's public IP.

Real-Life Analogy

Imagine an apartment building.

Public IP = Building address

15 MG Road, Bangalore

Private IP = Apartment numbers

- Flat 101
- Flat 102
- Flat 103

The courier first reaches the building (public IP), then the correct apartment (private IP).

What is a Router?

A **Router** connects different networks and forwards data between them.

At home:

Internet

    |

Router

├── Laptop

├── Phone

└── TV

The router:

- Connects your home network to the Internet.
- Sends data to the correct device.

What is a Switch?

A **Switch** connects devices **within the same local network (LAN)**.

Example:

Office

Computer A

Computer B

Computer C

    |

  Switch

The switch helps local devices communicate efficiently.

Router vs Switch

Router	Switch
Connects different networks	Connects devices in the same network
Works with IP addresses	Primarily uses MAC addresses to forward frames
Connects to the Internet	Connects local devices

What is DNS?

Imagine you want to visit:

www.google.com

Computers don't understand names.

They communicate using IP addresses.

DNS (Domain Name System) translates:

www.google.com

↓

142.250.xxx.xxx

It's like a phone contacts list.

Instead of remembering phone numbers, you remember names.

DNS does the same thing for websites.

Complete Flow

When you type:

www.amazon.in

Here's what happens:

1. Browser

↓

2. DNS finds the IP address

↓

3. Internet sends the request

↓

4. Amazon Server receives it

↓

5. Amazon sends the webpage

↓

6. Browser displays the page

This process usually takes only milliseconds.

Summary

Today you learned:

- What a network is
- LAN, MAN, and WAN
- What the Internet is
- What an IP address is

- Public vs Private IP
- Router
- Switch
- DNS

These concepts form the foundation for understanding AWS networking.

Interview Questions

1. What is a network?
2. What is an IP address?
3. Why does every device need an IP address?
4. What is the difference between a public IP and a private IP?
5. What is a router?
6. What is a switch?
7. What is DNS?
8. What happens when you type www.google.com into a browser?

Practice Exercise

Suppose you are at home with:

- 1 Wi-Fi router
- 1 Laptop
- 2 Mobile phones
- 1 Smart TV

Answer these questions:

1. Which device connects your home network to the Internet?
2. Which devices have private IP addresses?
3. Which device usually has the public IP address?
4. What role does DNS play when you open www.youtube.com?

Lesson 8: Ports, HTTP, HTTPS, TCP, UDP, Firewalls, NAT, and Gateways

Before We Start

Let's understand a simple scenario.

Imagine your laptop has one IP address:

192.168.1.10

Now imagine you're running:

- Chrome
- WhatsApp
- Zoom
- Spotify

All of them are using the same IP address.

Question:

If all applications use the same IP address, how does the computer know which application should receive incoming data?

The answer is **Ports**.

---

1. What is a Port?

A **port** is a logical communication endpoint used by applications on a computer.

Think of an IP address as an apartment building.

Building Address

15 MG Road

Bangalore

Inside the building there are many apartments.

Flat 101

Flat 102

Flat 103

Flat 104

The courier first reaches the building, then delivers the package to the correct apartment.

Similarly:

- **IP Address = Building**
- **Port = Apartment Number**

The IP identifies the computer.

The port identifies the application.

Example

Your laptop:

IP Address

192.168.1.10

Applications:

Chrome → Port 443

Node.js App → Port 3000

MySQL → Port 3306

SSH → Port 22

All applications share the same IP but use different ports.

Common Port Numbers

Port	Service
20, 21	FTP
22	SSH
23	Telnet

Port	Service
25	SMTP
53	DNS
80	HTTP
110	POP3
143	IMAP
443	HTTPS
3306	MySQL
5432	PostgreSQL
6379	Redis
27017	MongoDB

💡 **Interview Tip:** You don't need to memorize every port initially, but **22, 53, 80, 443, 3306, and 5432** are very commonly asked.

Real Example

You type:

https://www.google.com

The browser connects to:

Google Server

IP Address

↓

Port 443

Because HTTPS uses **port 443**.

---

2. HTTP

HTTP stands for:

HyperText Transfer Protocol

It is a protocol used for communication between:

- Browser
- Web Server

Example:

Browser

↓

HTTP Request

↓

Web Server

↓

HTTP Response

↓

Browser

Problem with HTTP

HTTP sends data as **plain text**.

Example:

Username

preethi

Password

123456

Anyone intercepting the traffic could read it.

This is insecure.

---

3. HTTPS

HTTPS stands for:

HyperText Transfer Protocol Secure

HTTPS encrypts data before sending it.

Username

Encrypted

Password

Encrypted

Now even if someone captures the traffic, they cannot easily understand it.

HTTP vs HTTPS

HTTP	HTTPS
Not encrypted	Encrypted

HTTP	HTTPS
Port 80	Port 443
Less secure	More secure
Suitable for non-sensitive data	Used for login, banking, shopping

---

Real Example

HTTP

http://example.com

Anyone on the network could potentially read the transmitted data.

HTTPS

https://amazon.in

Payment details and passwords are encrypted before transmission.

---

4. TCP

TCP stands for:

Transmission Control Protocol

TCP is:

- Reliable
- Connection-oriented
- Ensures data arrives correctly and in order

Real-Life Example

Imagine sending an important legal document by courier.

You expect:

- Delivery confirmation
- No missing pages

- Correct order

TCP works like that.

TCP Features

- Reliable delivery
- Error checking
- Retransmits lost packets
- Maintains packet order

Where TCP is Used

- Websites
- Banking
- Email
- APIs
- File downloads

---

5. UDP

UDP stands for:

User Datagram Protocol

UDP is:

- Faster
- Connectionless
- Does not guarantee delivery or order

Real-Life Example

Imagine a live cricket commentary over a loudspeaker.

If one word is missed, the commentary continues.

No one stops to replay the missed word.

UDP behaves similarly.

UDP is Used In

- Online games
- Live video
- Voice calls
- Video conferencing
- Live streaming

TCP vs UDP

TCP	UDP
Reliable	Best effort
Slower	Faster
Connection-oriented	Connectionless
Guarantees delivery	No delivery guarantee
Maintains order	Order not guaranteed

Which One Does Netflix Use?

Netflix often uses **TCP** because video segments are typically delivered over HTTP/HTTPS, which runs on TCP. Real-time voice and video applications (like video calls) commonly use UDP-based protocols because low latency is more important than perfect reliability.

---

6. Firewall

A **Firewall** acts like a security guard for a network or computer.

It decides:

- Allow traffic

- Block traffic

Example

Imagine your office security guard.

Employees:

✅ Allowed

Unknown visitor:

❌ Blocked

A firewall performs a similar role for network traffic.

Firewall Example

Internet

↓

Firewall

↓

Server

Only approved traffic reaches the server.

---

7. NAT (Network Address Translation)

Imagine your home has:

- Laptop
- Phone
- Smart TV

Each device has a private IP.

Laptop

192.168.1.10

Phone

192.168.1.11

TV

192.168.1.12

But your Internet Service Provider gives your router only **one public IP**.

NAT allows all your private devices to share that single public IP when accessing the internet.

NAT Diagram

Laptop

Phone

TV

↓

Router (NAT)

↓

Public IP

↓

Internet

---

8. Gateway

A **Gateway** is the device or service that allows one network to communicate with another network.

Your home router acts as the gateway between:

- Your home network
- The Internet

Complete Request Flow

Imagine you open:

https://www.amazon.in

Here's what happens:

Browser

↓

DNS finds Amazon's IP

↓

Router

↓

Internet

↓

Firewall

↓

Amazon Server

↓

Port 443 (HTTPS)

↓

Amazon Application

↓

Response returns

↓

Browser displays the page

Summary

Concept	Meaning
Port	Identifies the application on a device
HTTP	Web communication without encryption
HTTPS	Secure web communication with encryption
TCP	Reliable communication
UDP	Fast communication with no delivery guarantee
Firewall	Filters incoming and outgoing traffic
NAT	Lets many private devices share one public IP
Gateway	Connects different networks

Interview Questions

1. What is a port?
2. Why are ports needed?
3. What is the difference between HTTP and HTTPS?
4. What are the default ports for HTTP and HTTPS?
5. Explain TCP and UDP.
6. Which protocol is used for web browsing?
7. What is a firewall?
8. What is NAT?
9. What is a gateway?

Practice Exercise

Suppose you visit:

https://chat.openai.com

Can you explain the complete journey?

1. What converts the website name into an IP address?

2. Which device sends your traffic from your home to the internet?
3. Which port is typically used?
4. Which protocol carries the web traffic?
5. Why is HTTPS preferred over HTTP?
6. What role does the firewall play on the server side?

If you can answer these questions, you've built a strong networking foundation.

Congratulations! 🎉

At this point, you have the core knowledge required to begin learning **AWS**. You now understand:

- ✅ What cloud computing is
- ✅ Why cloud computing exists
- ✅ The five essential cloud characteristics
- ✅ IaaS, PaaS, and SaaS
- ✅ Data centers, regions, and availability zones
- ✅ Virtualization and hypervisors
- ✅ Networking fundamentals
- ✅ Ports, protocols, firewalls, NAT, and gateways

Next Lesson: Introduction to AWS

We'll start using the cloud in practice by learning:

- What is AWS?
- Why AWS is the market leader
- AWS Global Infrastructure
- AWS Management Console
- AWS Free Tier
- AWS Account Setup
- AWS Core Services Overview

From there, we'll move into hands-on services like **IAM**, **EC2**, **S3**, **VPC**, **RDS**, and **Lambda**.

Lesson 9: Introduction to AWS

Learning Objectives

By the end of this lesson, you will understand:

- What is AWS?
- Why AWS is so popular
- Who uses AWS?
- AWS Global Infrastructure
- AWS Services Overview
- AWS Free Tier
- AWS Console
- How AWS works

What is AWS?

AWS (Amazon Web Services) is a cloud computing platform provided by Amazon.

It allows individuals and companies to rent cloud resources such as:

- Virtual Servers
- Storage
- Databases
- Networking
- AI Services
- Machine Learning
- Security
- Monitoring
- Analytics

Instead of buying hardware, you rent these services over the Internet.

A Simple Example

Suppose you create an online shopping website.

Without AWS, you would need to buy:

Server

CPU

RAM

Storage

Networking Equipment

Cooling System

UPS

Backup System

This could cost lakhs of rupees.

With AWS:

Create AWS Account

↓

Launch EC2 Server

↓

Upload Website

↓

Website is Live

This can often be done in minutes.

Why Was AWS Created?

Amazon originally built infrastructure for its own online store.

They realized they had created powerful tools for:

- Computing
- Storage
- Databases
- Networking

Instead of keeping them only for Amazon, they made them available to everyone as cloud services.

That became **Amazon Web Services (AWS)**.

Why is AWS Popular?

AWS is popular because it provides:

- Hundreds of cloud services
- Global infrastructure
- Strong security features
- High reliability
- Elastic scaling
- Pay-as-you-go pricing
- Extensive documentation
- A large ecosystem and community

Companies Using AWS

Many well-known companies use AWS for at least part of their infrastructure, including:

- Netflix
- Airbnb
- Samsung

- Adobe
- Slack
- Pinterest
- Twitch

Different organizations use different cloud providers or a combination of them, but AWS is one of the most widely adopted.

AWS Global Infrastructure

AWS has infrastructure around the world.

World

├── North America
├── South America
├── Europe
├── Asia Pacific
├── Middle East
└── Africa

Each geographical area contains one or more **AWS Regions**.

Region Example

Asia Pacific

├── Mumbai
├── Hyderabad
├── Singapore
├── Tokyo
├── Seoul
└── Sydney

Each Region contains multiple **Availability Zones (AZs)**.

Availability Zones

Example:

Mumbai Region

├── AZ-A
├── AZ-B
└── AZ-C

Each Availability Zone contains one or more physical data centers.

This design improves reliability and fault tolerance.

AWS Global Hierarchy

World

↓

Region

↓

Availability Zone

↓

Data Center

↓

Physical Servers

↓

Virtual Machines (EC2)

This is exactly where your applications eventually run.

AWS Services

AWS provides more than 200 cloud services.

Here are the most important ones:

Service	Purpose
IAM	User and access management
EC2	Virtual servers
S3	Object storage
EBS	Block storage for EC2
RDS	Managed relational databases
VPC	Private networking
Route 53	DNS service
CloudFront	Content Delivery Network (CDN)
Lambda	Serverless computing
API Gateway	API management
CloudWatch	Monitoring and logging

You don't need to learn all of them at once. We'll study them one by one.

AWS Free Tier

AWS offers a **Free Tier** so beginners can learn many services without paying, subject to usage limits.

For example, many common learning activities can be done within the free tier limits if you stay under the monthly quotas.

⚠️ **Important:** If you exceed the free tier limits or use services that aren't included, charges may apply.

AWS Management Console

The **AWS Management Console** is the web interface where you manage AWS resources.

Example workflow:

Login

↓

Open EC2

↓

Launch Instance

↓

Connect to Server

↓

Deploy Website

Most beginners start with the console before learning automation.

AWS CLI (Command Line Interface)

Instead of clicking buttons, you can manage AWS using commands.

Example:

aws ec2 describe-instances

This lists your EC2 instances.

Later, you'll also learn **Infrastructure as Code** using tools like Terraform and AWS CloudFormation.

AWS SDK

If you want your application to interact with AWS programmatically, you use an AWS SDK.

For example, a Python application can upload a file to S3 using the AWS SDK for Python (boto3).

How AWS Works

Let's follow a real example.

Suppose you launch a website on EC2.

User

↓

Browser

↓

Internet

↓

AWS Region (Mumbai)

↓

Availability Zone

↓

EC2 Server

↓

Website

↓

Response

↓

Browser

Notice how all the concepts you've already learned fit together:

- Networking
- DNS

- Regions
- Availability Zones
- Virtualization
- Virtual Machines

AWS Core Services Roadmap

We'll learn these in order:

AWS

↓

IAM

(User Management)

↓

EC2

(Virtual Server)

↓

EBS

(Storage)

↓

S3

(Object Storage)

↓

VPC
(Networking)

↓

RDS
(Database)

↓

Load Balancer

↓

Auto Scaling

↓

CloudWatch

↓

Lambda

↓

API Gateway

This sequence builds from the basics to more advanced architectures.

Real-World Example

Imagine you're building an online shopping website.

You might use:

- **IAM** → Manage users and permissions
- **EC2** → Host your application
- **S3** → Store product images
- **RDS** → Store customer and order data
- **VPC** → Secure your network
- **CloudWatch** → Monitor application health
- **Route 53** → Map your domain name (e.g., www.myshop.com)
- **CloudFront** → Deliver images quickly to users worldwide

Each AWS service has a specific role.

Summary

Today you learned:

- What AWS is
- Why AWS is popular
- AWS global infrastructure
- Regions and Availability Zones
- The AWS Management Console
- AWS CLI and SDK

- The major AWS services you'll learn

Interview Questions

1. What is AWS?
2. Why is AWS popular?
3. What is an AWS Region?
4. What is an Availability Zone?
5. What is the AWS Free Tier?
6. What is the AWS Management Console?
7. What is the difference between the AWS Console, CLI, and SDK?
8. Name five important AWS services and explain their purpose.

Practice Exercise

Imagine you're creating a photo-sharing application.

Answer these questions:

1. Which AWS service would you use to store uploaded photos?
2. Which service would host your backend application?
3. Which service would store user account information?
4. Which service would monitor server performance?
5. Which service would manage who can access your AWS account?

Try answering these before moving on

Next Lesson: AWS IAM (Identity and Access Management)

What is IAM?

IAM is the AWS service that controls **who can access AWS resources and what they are allowed to do**.

Think of IAM as the **security guard** for your AWS account.

Real-Life Example

Imagine a company.

- **CEO** → Can access everything.
- **Manager** → Can access only their department.
- **Employee** → Can access only their assigned work.

AWS IAM works the same way.

IAM Components

1. **User** 👤
    - Represents one person.
    - Example: Preethi
2. **Group** 👥
    - A collection of users with the same permissions.
    - Example: Developers, Admins
3. **Policy** 📜
    - A JSON document that defines permissions.
    - Example:
        - Can create EC2
        - Can read S3
        - Cannot delete RDS
4. **Role** 🎭

- Temporary permissions assigned to AWS services or users.
- Example: An EC2 instance gets permission to access an S3 bucket without storing AWS keys.

Root User vs IAM User

Root User	**IAM User**
Created when AWS account is opened	Created by the root user or an admin
Full access to everything	Limited access based on permissions
Should be used rarely	Used for daily work

Best Practice:

- Use the **Root User** only for account-level tasks.
- Create **IAM Users** for everyday use.

Principle of Least Privilege

Give users **only the permissions they need**.

Example:

- Developer → Can launch EC2 instances.
- HR → Cannot launch EC2 instances.

This reduces security risks.

Interview Questions

1. What is IAM?
2. What is the difference between a Root User and an IAM User?
3. What is an IAM Role?
4. What is an IAM Policy?
5. What is the Principle of Least Privilege?

Next Topic

EC2 (Elastic Compute Cloud) – Learn how to create and manage virtual servers in AWS.

give next next next

Great! Here's a **short version** of the next AWS topics.

Lesson 11: EC2 (Elastic Compute Cloud)

What is EC2?

EC2 is AWS's **virtual server** service.

You can create a server in minutes without buying physical hardware.

Uses

- Host websites
- Run backend applications
- Install databases
- Run machine learning models

Example

You create:

- Ubuntu Server
- 2 vCPU
- 4 GB RAM
- 50 GB Storage

AWS creates the virtual machine for you.

Interview Question: What is EC2?

Lesson 12: EBS (Elastic Block Store)

What is EBS?

EBS is **hard disk storage** for EC2.

Think of it as the **SSD/HDD** attached to your virtual server.

Features

- Persistent storage
- Fast
- Can take snapshots (backups)

Example
EC2 = Computer

EBS = Hard Disk

Interview Question: What is the difference between EC2 and EBS?

Lesson 13: S3 (Simple Storage Service)

What is S3?

S3 is **object storage**.

It stores:

- Images
- Videos
- PDFs
- Backups
- Documents

Example

Instagram stores photos.

Those photos can be stored in S3.

Interview Question: What is S3 used for?

Lesson 14: VPC (Virtual Private Cloud)

What is VPC?

VPC is your **private network inside AWS**.

It lets you control:

- IP addresses

- Subnets
- Routing
- Security

Think of it as your own private office inside a large building.

Interview Question: What is VPC?

Lesson 15: RDS (Relational Database Service)

What is RDS?

RDS is AWS's **managed relational database service**.

Supported databases:

- MySQL
- PostgreSQL
- MariaDB
- Oracle
- SQL Server

AWS handles:

- Backups
- Updates
- Maintenance

You focus on your application.

Interview Question: Why use RDS instead of installing MySQL on EC2?

Lesson 16: Route 53

What is Route 53?

Route 53 is AWS's **DNS service**.

It converts:

www.amazon.com

↓

IP Address

Without Route 53, users would have to remember IP addresses.

Interview Question: What is Route 53?

Lesson 17: Load Balancer

What is a Load Balancer?

A Load Balancer distributes incoming traffic across multiple servers.

Users

  │

Load Balancer

├── EC2-1

├── EC2-2

└── EC2-3

Benefits:

- Prevents server overload
- Improves availability
- Increases performance

Interview Question: Why do we need a Load Balancer?

Lesson 18: Auto Scaling

What is Auto Scaling?

Auto Scaling automatically adds or removes EC2 instances based on traffic.

Example:

- 100 users → 1 EC2

- 10,000 users → 10 EC2
- 100 users again → Back to 1 EC2

Benefits:

- Saves money
- Handles traffic spikes automatically

Interview Question: What is Auto Scaling?

Lesson 19: CloudWatch

What is CloudWatch?

CloudWatch is AWS's **monitoring service**.

It monitors:

- CPU usage
- Memory (with additional setup)
- Disk
- Network traffic
- Logs

Example:
If CPU usage exceeds 80%, CloudWatch can trigger an alarm.

Interview Question: What is CloudWatch used for?

Lesson 20: AWS Lambda

What is Lambda?

Lambda lets you run code **without managing servers**.

You upload your code, and AWS runs it when triggered.

Example:

- Upload an image to S3.
- Lambda automatically resizes the image.

Benefits:

- No server management
- Pay only when your code runs

Interview Question: What is Serverless Computing?

AWS Learning Flow

IAM

↓

EC2

↓

EBS

↓

S3

↓

VPC

↓

RDS

↓

Route 53

↓

Load Balancer

↓

Auto Scaling

↓

CloudWatch

↓

Lambda

Next Step

Now that you have an overview, the best way to learn is to **deep dive into each service** with hands-on practice.

I recommend starting with **EC2**, because once you understand EC2, services like EBS, VPC, Load Balancer, and Auto Scaling become much easier to understand.

Scenario: Building an Amazon-like E-commerce Website

Users will:

- Sign up/Login
- Browse products
- View product images
- Place orders
- Make payments

Now let's see where each AWS service fits.

---

1. IAM (Identity and Access Management)

Purpose

Controls **who can access your AWS account and resources**.

Example

Your company has three employees:

Employee	Job	Permission
Preethi	Admin	Full Access
Rahul	Developer	Create EC2, Upload Code
Priya	Tester	View Resources Only

Using **IAM**, you create three users with different permissions.

Real Life

Think of a college.

- Principal → Can access everything
- Teacher → Can access classrooms
- Student → Can access only their classroom

IAM works the same way.

---

2. EC2 (Elastic Compute Cloud)

Purpose

Runs your application.

Example

Suppose your website is built using:

- React (Frontend)
- Node.js (Backend)

You launch an EC2 server.

Internet



EC2 Server

Ubuntu

Node.js

Express

React

Now when users visit your website:

www.myshop.com

↓

EC2

↓

Website Opens

EC2 is like the **computer that runs your application**.

---

3. EBS (Elastic Block Store)

Purpose

Stores the operating system and application files of EC2.

Example

Your EC2 server contains:

Ubuntu OS

Node.js

Project Folder

Logs

Configuration Files

All these are stored inside EBS.

Think of:

EC2 = Laptop

EBS = Laptop SSD

Without EBS, your EC2 would have nowhere to store files.

---

4. S3 (Simple Storage Service)

Purpose

Stores files.

Example

Your shopping website has:

- Product Images
- Product Videos
- User Profile Pictures
- PDF Invoices

Instead of storing these on EC2, you store them in S3.

Customer

↓

Opens Product

↓

Image comes from S3

Example:

Laptop Image

↓

S3 Bucket

↓

Website Shows Image

S3 is like **Google Drive for your application**.

---

5. RDS (Relational Database Service)

Purpose

Stores structured data.

Example

Customer registers.

Name

Email

Password

Phone Number

These details are stored in RDS.

Orders are also stored.

Customer ID

Product ID

Price

Order Date

Images go to S3.

Customer information goes to RDS.

---

6. Route 53

Purpose

Converts website names into IP addresses.

Example

Customer types:

www.myshop.com

Route 53 finds:

54.221.10.25

Then sends the request to the correct server.

Without Route 53:

Customers would have to remember:

54.221.10.25

Instead of

www.myshop.com

---

7. Load Balancer

Suppose:

100,000 people visit your website.

Without Load Balancer

100000 Users

↓

One EC2

↓

Crash

With Load Balancer

100000 Users

↓

Load Balancer

↓

EC2-1

EC2-2

EC2-3

EC2-4

The traffic is divided among all servers.

Every server gets fewer requests.

The website stays fast.

---

8. Auto Scaling

Suppose today is Diwali Sale.

Traffic becomes:

500 Users

↓

50000 Users

Auto Scaling automatically creates more EC2 servers.

Before

EC2-1

After

EC2-1

EC2-2

EC2-3

EC2-4

EC2-5

After the sale:

AWS removes the extra servers automatically.

This saves money.

---

9. CloudWatch

CloudWatch monitors your servers.

Example

CPU Usage:

30%

40%

90%

CloudWatch notices:

CPU > 80%

It sends an alert.

Or it tells Auto Scaling:

Launch Another EC2

CloudWatch is like a **doctor** checking the health of your servers.

---

10. Lambda

Suppose a customer uploads a profile picture.

Instead of a person resizing the image:

Upload Image

↓

S3

↓

Lambda Triggered

↓

Resize Image

↓

Store Back in S3

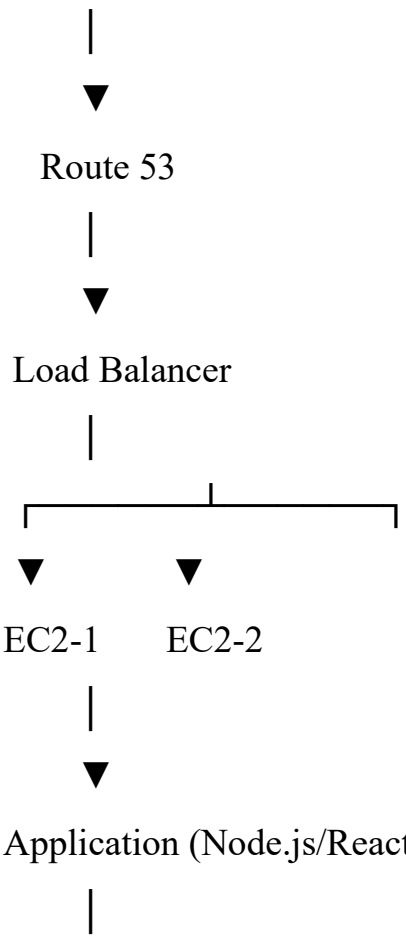
No server is required.

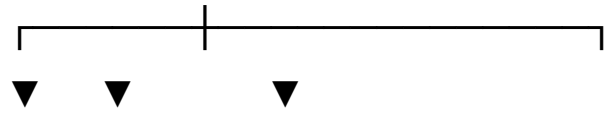
Lambda runs only when needed.

Complete Flow of an Amazon-like Website

Customer Opens

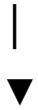
www.myshop.com





RDS S3 CloudWatch

(DB) (Images) (Monitoring)



Lambda

(Image Resize)

Auto Scaling adds/removes EC2 servers based on traffic.

IAM controls who can manage these AWS resources.

Easy Memory Trick

Think of AWS services like building a shopping mall:

AWS Service	Real-Life Example
IAM	Security Guard (controls entry)
EC2	Shop (where business happens)
EBS	Store Room (keeps files)
S3	Warehouse (stores products/files)
RDS	Customer Register (stores records)
Route 53	Address Book (finds the shop)
Load Balancer	Traffic Police (distributes visitors)
Auto Scaling	Opens more checkout counters during rush
CloudWatch	CCTV + Security Monitor
Lambda	Automatic Door (works only when someone arrives)

This is how a real application works.

When you become a **Cloud Engineer, Backend Developer, or DevOps Engineer**, you'll use these services together to build scalable and reliable applications.

DevOps on Cloud (Complete Beginner to Intermediate Notes)

What is DevOps?

DevOps is a combination of two words:

- **Dev** = Development (Developers who write code)
- **Ops** = Operations (Engineers who deploy and maintain applications)

DevOps is a **culture, process, and set of tools** that helps developers and operations teams work together to **build, test, deploy, and maintain software quickly, reliably, and automatically**.

Goal of DevOps

- Faster software delivery
- Better collaboration
- Fewer manual errors
- Continuous improvement
- Reliable deployments

Before DevOps

Imagine a company developing an **Amazon-like shopping website**.

Step 1

Developer writes the code.

Login Page

Shopping Cart

Payment Module

↓

Step 2

Developer sends the code to the Operations team.

↓

Step 3

Operations manually:

- Copy files
- Install software
- Configure servers
- Start the application

↓

Problems

- Slow deployment
- Human mistakes
- Poor communication
- Long downtime if something goes wrong

After DevOps

Everything becomes automated.

Developer

|

Writes Code

|

Git

|

GitHub

|

CI/CD Pipeline (Jenkins)

|

Docker

|

Kubernetes

|

Terraform

Ansible

AWS Cloud

Users Access Website

Now every step is automated, faster, and more reliable.

DevOps Workflow (Step by Step)

Step 1: Developer Writes Code

A developer creates an application.

Example:

- React Frontend
- Node.js Backend
- MySQL Database

The code is written on the developer's laptop.

Step 2: Git (Version Control)

What is Git?

Git is a **Version Control System (VCS)**.

It tracks every change made to your source code.

Why Git?

Suppose you accidentally delete an important feature.

Without Git:

- You may lose your work.

With Git:

- You can restore an older version.

Example

Version 1

Login Page

↓

Version 2

Login + Registration

↓

Version 3

Payment Feature

↓

Version 4

Bug Introduced

↓

Git can restore Version 3.

Real-Life Example

Think of Git as **Microsoft Word's Version History**.

You can always go back to an earlier version.

Step 3: GitHub

What is GitHub?

GitHub is an online platform that stores Git repositories.

It allows multiple developers to work on the same project.

Example

Developer A (India)

|

Push Code

|

GitHub

|

Pull Code

|

Developer B (USA)

Everyone works on the latest version of the project.

Why GitHub?

- Backup code
- Team collaboration
- Code reviews
- Project management
- Integration with CI/CD

Step 4: CI/CD

What is CI?

CI = **Continuous Integration**

Developers regularly merge their code into a shared repository.

Each time code is pushed:

- Build automatically
- Run tests automatically
- Detect errors early

Example

Developer Pushes Code

↓

GitHub

↓

Automatic Build

↓

Automatic Tests

↓

Ready for Deployment

What is CD?

CD = **Continuous Delivery / Continuous Deployment**

After testing succeeds, the application is automatically deployed.

Example

Push Code

↓

Tests Pass

↓

Deploy to AWS

↓

Website Updated

No manual deployment.

Step 5: Jenkins

What is Jenkins?

Jenkins is an **automation server** used to create CI/CD pipelines.

It automates:

- Build
- Test
- Deploy

Example

Developer pushes code.

Jenkins automatically:

Download Code

↓

Compile

↓

Run Tests

↓

Create Docker Image

↓

Deploy to AWS

No manual work is required.

Step 6: Docker

What is Docker?

Docker is a platform used to package an application and all its dependencies into a **Container**.

Why Docker?

Without Docker:

Developer Laptop

Ubuntu

Node.js 22

MongoDB Installed

Works ✅

Production Server

Windows

Node.js 18

MongoDB Missing

Fails ❌

This is known as the **"It works on my machine"** problem.

Docker Solution

Docker packages:

Application

Node.js

Libraries

Dependencies

Configuration

Everything needed to run the application is packaged together.

What is a Container?

A **Container** is a lightweight package that contains:

- Application
- Runtime

- Libraries
- Dependencies
- Configuration

The container can run on:

- Laptop
- Testing Server
- AWS
- Azure
- Google Cloud

The behavior remains the same.

Docker Image vs Docker Container

Docker Image

A **Docker Image** is a blueprint or template.

Example:

Ubuntu + Node.js + Shopping App

It is **not running**.

Docker Container

A **Docker Container** is the **running instance** of a Docker Image.

Easy Analogy

Recipe

↓

Cake

Docker Image

↓

Docker Container

Image = Blueprint

Container = Running Application

Step 7: Kubernetes

What is Kubernetes?

Kubernetes (K8s) is a **container orchestration platform**.

It manages Docker containers automatically.

Responsibilities

- Start containers
- Stop containers
- Restart failed containers
- Scale containers
- Balance traffic

Example

Normal Traffic

Container 1

Festival Sale

Container 1

Container 2

Container 3

Container 4

Container 5

Kubernetes creates more containers automatically.

Step 8: Terraform

What is Terraform?

Terraform is an **Infrastructure as Code (IaC)** tool.

Instead of manually creating cloud resources, you write code describing the infrastructure.

Example

Terraform file:

Create:

2 EC2 Instances

1 VPC

1 RDS Database

1 Load Balancer

Run Terraform.

AWS automatically creates all these resources.

Benefits

- Repeatable
- Faster
- Consistent
- Easy to recreate infrastructure

Step 9: Ansible

What is Ansible?

Ansible is a **Configuration Management and Automation Tool**.

It installs software and configures servers automatically.

Example

Suppose you have **100 EC2 servers**.

Without Ansible:

Login Server 1

Install Node.js

Login Server 2

Install Node.js

...

Repeat 100 Times

Very time-consuming.

With Ansible:

Run One Playbook

↓

All 100 Servers

Install Node.js

Install Nginx

Copy Application

Start Service

Everything is automated.

Complete DevOps Architecture

Developer

Write Code

Git

GitHub

Jenkins (CI/CD)

Docker (Package Application)



Docker Image



Docker Container



Kubernetes (Manage Containers)



Terraform (Create AWS Infrastructure)



Ansible (Configure Servers)



AWS Cloud



Users Access Website

How Everything Works Together (Real Example)

Imagine you're updating your shopping website with a **"Wishlist"** feature.

1. The developer writes the new code.
2. **Git** tracks the code changes.
3. The code is pushed to **GitHub**.
4. **Jenkins** detects the change, builds the project, and runs tests.
5. If tests pass, Jenkins creates a **Docker Image**.
6. A **Docker Container** starts from that image.
7. **Kubernetes** deploys the container and scales it if traffic increases.
8. **Terraform** creates or updates the required AWS infrastructure.
9. **Ansible** installs software and configures the servers automatically.
10. Users see the new Wishlist feature on the live website.

Quick Summary

Tool	**Purpose**	**Real-Life Analogy**
Git	Track code changes	Version history in Word
GitHub	Store and share code	Google Drive for code
CI	Automatically build and test code	Quality check before shipping
CD	Automatically deploy code	Automatic delivery service
Jenkins	Automates CI/CD	Factory manager
Docker	Packages applications	Shipping container
Docker Image	Blueprint of the application	Recipe
Docker Container	Running application	Cooked meal
Kubernetes	Manages containers	Apartment building manager

Tool	Purpose	Real-Life Analogy
Terraform	Creates cloud infrastructure using code	Architect's building plan
Ansible	Configures and updates servers	Electrician setting up every room automatically

One-Line Memory Trick

Developer → Git → GitHub → Jenkins → Docker → Kubernetes → Terraform → Ansible → AWS → Users